BB BarberBoost

MOBILE STRATEGY DOCUMENT

Mobile PWA Roadmap

A complete strategy for transforming BarberBoost into an installable mobile experience for barbers and customers on both Android and iOS — without building a new codebase.

PREPARED FOR	DATE	VERSION	STATUS
Dan Jatau	May 2026	1.0	Approved for Implementation

SECTION 01

Approach Options

Three viable paths were evaluated for delivering BarberBoost on Android and iOS. Each was assessed against the criteria of development speed, code reuse, and native experience quality.

Option 1 — Progressive Web App (PWA)

RECOMMENDED LOWEST COMPLEXITY

Enhance the existing Next.js application to be installable on home screens like a native app. No new codebase is required — everything already built carries over. A `manifest.json`, service worker, and web push notifications are added on top of the current app.

> **Trade-off:** No App Store or Google Play listing. Android (Chrome) has excellent PWA support including push notifications and install prompts. iOS 16.4+ supports push notifications via PWA; older iOS versions do not. Some users won't know how to install without a store link — mitigated by an in-app install prompt on both platforms.

Option 2 — Capacitor

MIDDLE GROUND

Wraps the existing Next.js web app in a native shell and publishes to both the Apple App Store and Google Play Store. The codebase remains unchanged — Capacitor provides a bridge for native APIs including push notifications on both platforms.

> **Trade-off:** UI is rendered in a WebView so it won't feel fully native on either platform, but it achieves store presence on both. Approximately 1–2 days of configuration. A natural upgrade path from the PWA if store listings become a business requirement.

Option 3 — Expo (React Native)

V2 RECOMMENDATION

A fully native application using the existing API routes as its backend. All UI components are rewritten in React Native. SDKs for Supabase and Stripe are available. Approximately 70% of business logic (hooks, types, API calls) can be shared.

> **Trade-off:** Requires building an entirely new frontend. Recommended as V2 once V1 PWA has validated mobile demand and revenue impact.

SECTION 02

V1 Feature Priorities

The V1 release targets both barbers and customers simultaneously, with a hard scope boundary to avoid shipping two half-finished products. Build order: Barber features first, then Customer features.

Barber / Shop Owner

Build first — drives subscription retention

- **Today's Schedule**
  Clean list of the day's bookings. The #1 reason a barber opens the app.
- **Booking Status Updates**
  Mark as completed, no-show, or cancelled without a laptop.
- **New Booking Push Notification**
  Instant alert when a customer books. Critical for shops not at a desk.
- **Quick Client Lookup**
  Search by name, view visit history and contact info.
- **Today's Stats Glance**
  Bookings today and revenue today. Single screen.
- **Inventory Management** V2
  Deferred to V2
- **AI Marketing Copy** V2
  Deferred to V2 (Empire plan only)

Customer

Build second — drives bookings to barbers

- **Book an Appointment**
  Service → Staff → Time → Confirmation. The entire value proposition.
- **Shop Direct Link**
  Existing /booking/[slug] flow. Most customers arrive via a shared link.
- **My Bookings**
  View upcoming and past appointments via lightweight guest lookup.
- **Appointment Reminder Push**
  "Your appointment is tomorrow at 2pm." Justifies installing the app.
- **Shop Search / Map Browse** V2
  Requires significant new backend work
- **In-App Payments** V2
  Adds App Store review complexity
- **Loyalty / Rewards** V2
  Not in current system

SECTION 03

Mobile Responsiveness Audit

Every key page and component was reviewed against small-screen behaviour. The codebase has stronger mobile foundations than typical Next.js dashboard apps — the shell, navigation, and most page layouts are already ready.

COMPONENT	STATUS	FINDING
Dashboard Shell	Ready	BottomNav with safe-area insets, mobile sidebar drawer with backdrop, `pb-20` content padding to clear nav.
Bottom Navigation	Ready	5 primary destinations, 44px minimum hit targets, `env(safe-area-inset-bottom)` for notched devices. Excellent.
Header	Ready	Hamburger menu on mobile, page title truncates, "New Booking" collapses to icon-only on small screens.
Dashboard Page	Ready	All stat grids use `grid-cols-1 sm:grid-cols-2 lg:grid-cols-4`. Chart row stacks on mobile via `grid-cols-1 lg:grid-cols-5`.
Bookings Page	Ready	Week view hidden on mobile (`hidden sm:contents`). List and Day views are fully responsive. Controls collapse to icon buttons.
Clients Page	Ready	Responsive grid, horizontally scrollable tag filters, full-width search on mobile, list header hidden on small screens.
Public Booking Page	Ready	Explicitly mobile-first. Sticky "Book Now" bar on mobile, `max-w-2xl` centred layout, scrollable horizontal carousels. Best-in-class.
Weekly Calendar (Dashboard)	Needs Fix	Uses `min-w-[600px]` — technically scrollable but a 7-column time grid on a 390px phone is unusable. Should be hidden below `lg`, showing TodaySchedule only.
Analytics Page	Needs Fix	BusiestHoursHeatmap and StaffTable have fixed-width columns that overflow on small screens. Wrap in `overflow-x-auto` containers or provide mobile-collapsed variants.
PWA Infrastructure	Not Built	No `manifest.json`, no service worker, no install prompt, no push notification support. This is the primary gap between "mobile website" and "installable PWA".

SECTION 04

PWA Implementation Plan

Four sequential phases take BarberBoost from responsive website to installable PWA with push notifications. Every improvement benefits the web app simultaneously.

1

Installable — Manifest & Icons

~Half Day

→ Create `/public/manifest.json` with `#c9a84c`, `#0a0a0a`, `standalone` — name, short name, theme colour, background, display, consumed by both Android and iOS

→ Generate icon assets at required sizes: 192×192, 512×512, and a maskable variant (Android adaptive icons); plus Apple touch icon variants for iOS home screen

→ Add Android meta tags to `app/layout.tsx`: `theme-color`, manifest link, `mobile-web-app-capable`

→ Add iOS meta tags to `app/layout.tsx`: `apple-mobile-web-app-capable`, `apple-touch-icon`, `apple-mobile-web-app-status-bar-style`

→ Add an in-app install prompt using the `beforeinstallprompt` event (Android/Chrome) with a manual "Add to Home Screen" guide for iOS Safari

2

Offline Fallback — Service Worker

~Half Day

→ Install `next-pwa` (wraps Workbox, works with Next.js App Router)

→ Configure in `next.config.ts` — precaches static assets automatically

→ Add a simple `/offline` page rendered when the network is unavailable

→ Verify service worker registration and caching in Chrome DevTools

3

Mobile Layout Fixes

~1 Day

→ Hide the `BookingCalendar` below `lg` — show `TodaySchedule` only on mobile (single class change)

→ Wrap analytics heatmap and staff table in `overflow-x-auto` scroll containers

→ Audit inventory and settings pages for any additional overflow issues

→ Verify all pages across iPhone SE (375px) and standard iPhone (390px) viewports

4

Push Notifications

~2 Days

→ Generate VAPID key pair, store public/private keys as environment variables

→ Create `push_subscriptions` table in Supabase (device endpoint, keys, user/shop association)

→ Add `/api/push/subscribe` API route to save device subscriptions

→ Add subscription prompt component in DashboardShell — triggered on first mobile visit after install

→ Trigger push from Stripe webhook on new booking, and from booking API on customer confirmation

→ Customer reminder: trigger from existing cron reminder logic

IMPLEMENTATION SUMMARY

3.5	0	100%
Days total estimated effort	New codebases required	Existing API routes reused

SECTION 05

Next Steps

The following actions are ready to begin immediately. All implementation has been planned against the existing codebase and requires no architectural changes.

1

Phase 1 — Create the PWA manifest and iOS meta tags

Add `/public/manifest.json`, generate icons for both Android (192×192, 512×512 maskable) and iOS (Apple touch icon variants), and update `src/app/layout.tsx` with Android `theme-color` and iOS `apple-mobile-web-app-capable` meta tags. This makes the app installable on both platforms immediately.

2

Phase 2 — Add service worker with next-pwa

Install `next-pwa`, configure Workbox caching strategy in `next.config.ts`, and add an offline fallback page. Verify precaching and offline behaviour in Chrome DevTools.

3

Phase 3 — Fix the two mobile layout issues

Hide `BookingCalendar` on mobile screens (one-line fix), and wrap analytics charts in `overflow-x-auto` containers. Verify layout across 375px and 390px viewports.

4

Phase 4 — Implement push notifications

Set up VAPID keys, create the `push_subscriptions` database table, build the subscribe API route, add the in-app permission prompt, and wire push triggers into the booking and webhook flows.

5

V2 Decision — Evaluate Capacitor for App Store presence

After the PWA has been live for 4–6 weeks, review mobile usage metrics and subscriber feedback. If App Store listing becomes a business requirement, migrate to Capacitor — it wraps the same codebase with approximately 1–2 days of additional configuration.